CrossAudit: A Git-Native, Cross-Vendor Audit Loop for Agentic Science

Abstract

Autonomous "AI scientist" systems increasingly generate, execute, and review research. In the frontier systems we survey, the reviewing agent is an internal critic chosen by the same operator (often from the same model family or vendor as the generating agent) exposing supervision to self-preference bias and to blind spots shared across a common training pipeline; the supervision trace, moreover, usually lives in opaque platform logs rather than in an artefact a third party can replay. We present **CrossAudit**, a lightweight, infrastructure-minimal protocol for supervising autonomous research pipelines, built on three commitments. First, heterogeneity: every experiment increment produced by a generator agent is audited by an agent from a different model vendor, against a versioned, human-authored rulebook (the Constitution). Second, a git-native ledger: experiments, audit reports, verdicts, disputes and escalations are all commits, so the supervision history can be re-read, diffed, and cited by anyone. Third, graded human oversight: deterministic, model-free checks and hard inconsistencies gate the pipeline, judgement calls do not, and a bounded revision loop escalates unresolved blockers to a human principal.

We specify the protocol as eight invariants; describe a public reference implementation (GitHub Actions plus a few hundred lines of dependency-light Python) that *targets* them and implements a subset (deterministic checks, reply validation, receipts, anchored callbacks; receipt-verified fail-closed admission is specified but not yet enforced); and report a live deployment of a closely related variant in a computational-chemistry pipeline whose generator (Claude-based), auditor (GPT-lineage), and compute (cloud HPC) are mutually decoupled.

In an exploratory seeded-defect trial (30 synthetic increments, 43 defects; its intended blinding voided by a cross-vendor audit of our own repository, whose findings we adopt and commit alongside the data), a same-family auditor scored 38/43 at both scoring tiers with zero false-positive blockers, while a cross-vendor auditor scored 41/43 at the lenient tier but 37/43 at the strict tier and returned BLOCKED on all thirty increments; because recall under the frozen scorer rises with finding volume (the arms emitted 17, 63, and 113 findings), permutation-corrected agreement places the two model arms within noise at the lenient tier and reverses their order at the strict tier. The trial shows that vendors disagree, not that either is better; the strongest evidence in the repository is the committed record of cross-vendor audits of this paper itself. We argue that cross-vendor audit trails of this kind are a practical basis for accountable machine science, and one a single researcher can adopt today.

1 Introduction

[TODO: condense §1 of `paper/crossaudit.tex` to 1.2 pages.]

2 Related work and the same-source problem

[TODO: condense §2 to 1.0 page.]

3 The CrossAudit protocol

[TODO: condense §3 to 2.2 pages around Figure 1.]

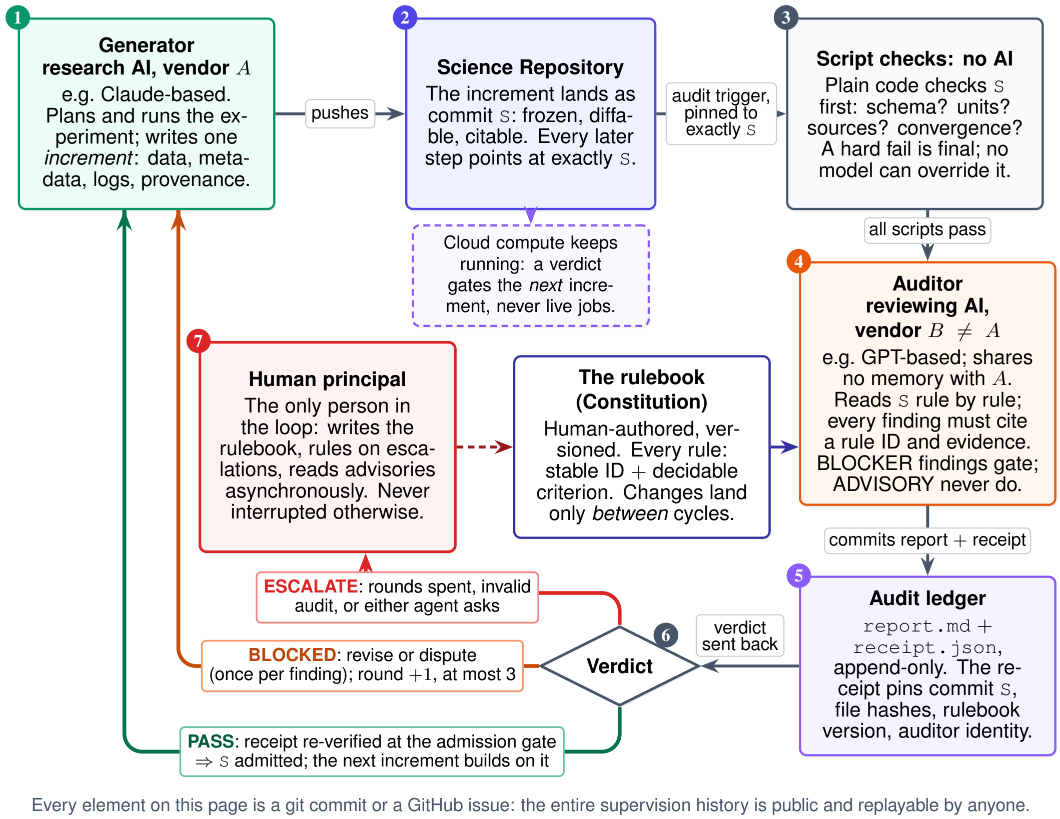

Figure 1: One experiment increment through CrossAudit. *[TODO: paste the full caption from crossaudit.tex and trim the invariant map if space demands.]*

4 Reference implementation, live deployment, and an audited trial

[TODO: condense §4 to 2.4 pages.]

5 Threat model and limitations

[TODO: condense §5 to 0.8 page; keep all five bullets.]

6 Discussion

[TODO: 0.9 page; keep CI-for-claims, ledger-as-data-asset, management-by-exception only.]

7 Conclusion

[TODO: 0.3 page; keep the homework close.]

A Full invariant statements

[TODO: paste the eight invariants verbatim from §3.2 if trimmed above.]

B Related-systems comparison

	Critic from a different vendor	Versioned, decidable rules	Replayable public ledger	Deterministic precedence	Bounded loop, escalation
The AI Scientist [**?**]	–	–	∼	–	∼
AI co-scientist [**?**]	–	–	–	–	∼
PoLL judge panels [**?**]	•	–	–	–	–
Cross-vendor review advice (§2)	•	–	∼	–	–
statcheck, PPS [**?**, **?**]	–	∼	∼	•	–
CrossAudit (this paper)	•	•	•	•	•

C Implementation status against the invariants

[TODO: tabulate the enforce-vs-target status from §4.1.]

D Seeded-defect trial scorecard

[TODO: paste the arm-comparison tabular and permutation floors.]